

문서를 LLM 위키로 바꿨다 — 무엇이 달라졌나

문서 6개를 노드 109개로 쪼개고, 원본·위키·규약의 세 층으로 나눴다.
결정 하나를 찾는 비용이 **1,928** → **1,263 토큰**으로 줄었고, 낡은 문서를 기계가 잡게 됐다.

기준일	2026년 7월 31일
근거	Andrej Karpathy, <i>LLM Wiki: A Pattern for Persistent Knowledge Bases</i>
범위	문서 체계만. 코드·스키마는 건드리지 않음
도구	Obsidian 볼트로 열림 · 스크립트 3종 추가

01 왜 바꿨나

문서가 늘면서 한 번 볼 때마다 전부를 읽어야 하는 구조가 됐다.

바꾸기 전의 문제

무엇이	왜 문제인가
DECISIONS.md 24,946 토큰	결정 56개가 한 파일에 시간순으로 쌓였다. 결정 하나는 중앙값 456토큰인데 실제로는 파일을 통째로 읽었다 — 이 작업 중 시스템이 "내용이 너무 커서 포함할 수 없다"며 요약에서 잘라냈을 정도다. grep 으로 구간만 읽는 방법이 있긴 했다(3장 B 경로). 다만 결정의 시작과 끝을 모르므로 읽을 범위를 짚어야 했고, 잘려도 알 수 없었다.
CLAUDE.md 10,031 토큰	에이전트가 매 세션 무조건 읽는 파일. BigQuery 설정처럼 특정 작업에만 필요한 내용이 항상 따라왔다.
뒤집힌 결정이 그대로 살아 있었다	힌트 요금을 순차 4단계에서 선택형으로 바꿨는데, 옛 결정이 같은 파일에 남아 있었다. 제목만 보면 서로 모순되는 항목도 있었다.
표 하나를 알려면 네 군데를 열어야	<code>heart_transaction</code> 은 DDL 주석 · 결정 6개 · 정합성 검사 5종 · 정책 7파일에 흩어져 있었다.
손으로 만든 목록이 조용히 낡았다	"비어 있는 테이블" 목록이 실제와 어긋나 있었다 — 20행짜리 표를 비었다고 적어두고, 정작 빈 표는 빠져 있었다.

공통점 — 다섯 가지 모두 **오류를 내지 않는다**. 문서는 잘 읽히고 링크도 멀쩡하다. 다만 낡았거나, 필요 없는 것까지 달려올 뿐이다. 이 프로젝트가 스키마에서 계속 상대해 온 "조용히 틀리는 것"이 문서에서도 똑같이 나타난 것이다.

02 지금의 3계층 구조

원본은 사람만 고치고, 위키는 에이전트가 소유하고, 규약이 그 일을 정의한다.

1계층 · raw/

원본

사람만 고침

바깥에서 들어온 사실. **고치지 않는다**. 회의에서 정한 것이 나중에 바뀌어도 회의록은 그대로 두고, 바뀐 것은 새 결정으로 적는다 — 그래야 "언제 왜 마음이 바뀌었나"가 남는다.

위치	개수	무엇
raw/meetings/		템플릿 팀 회의록. YYYY-MM-DD-주제.md
raw/legacy-analysis/	12	구 서비스 DB 분석 리포트 9종 + PDF 2종
raw/external/	0	API 스펙 등 외부 문서

⚠ legacy-analysis 는 구 서비스(다른 스키마)를 분석한 것이다. 행동 수치(이탈률·전환율)는 참고하되 표 구조는 우리 것이 아니다 — 그건 우리가 갈아엎은 대상이다.

2계층 · docs/

위키

에이전트가 소유

원본과 코드에서 **뽑아 만든 것**. 노드 하나가 오피시디언 노드 하나다.

위치	개수	무엇
docs/decisions/	56	결정 하나가 한 파일. status · superseded_by 로 관계 표시
docs/tables/	40	표마다 한 장. DDL·결정·검사·정책을 모아 자동 생성
docs/ops/	5	실제 작업할 때 필요한 값과 절차
docs/index.md	1	위키 전체 목록. 자동 생성
docs/log.md	1	무엇을 언제 했나. 덧붙이기만
DECISIONS.md	1	결정 색인. 기존 링크 19곳이 그대로 살도록 남김

3계층 · CLAUDE.md

규약

사람 + 에이전트

위키가 어떻게 생겼고 **어떻게 다루는**지를 정의한다. 세 연산이 절차로 적혀 있다.

연산	무엇을 하나
ingest	원본을 읽어 위키에 반영. 결정은 노드로, 뒤집으면 옛 노드에 표시, 색인 갱신, 원본을 ingested: true 로, 이력에 한 줄
query	전문을 훑지 않고 색인에서 고른 노드만 읽는다. 답이 다시 쓰일 것 같으면 그 답을 새 노드로 남긴다
lint	문서 주장을 살아 있는 DB·파일시스템과 대조 (db/doc_lint.py)

03 얻은 것 — 숫자

토큰 절감이 가장 크고, 그다음이 낮음 탐지다.

결정 하나를 찾는 데 드는 비용 — 다섯 경로

질문 5개(하트 경제·친구 끊기·합성 데이터·증분 함정·힌트 요금)로 실측한 평균이다. 읽기 호출 오버헤드를 포함했다.

경로	방법	토큰	비고
A	파일을 통째로 읽는다	25,066	쪼개기 전에 실제로 하던 것
B	grep 으로 줄을 찾아 그 구간만	1,928	⚠ 쪼개기 전에도 가능했다
C	색인을 읽고 노드 하나	2,656	색인이 1,810 이라 B 보다 무겁다
D	grep 으로 파일명만 받고 정확한 노드만	1,263	쪼개기 뒤 실제로 하게 되는 것
E	매칭된 노드를 전부 읽는다	6,907	⚠ B 보다 2.6배 나쁘다

공정한 비교는 B 대 D 다 — 절감은 34%.

처음에 "91% 절감"이라 적었던 것은 A(통째로 읽기)를 기준으로 삼은 것인데, 그건 게으른 선택이었지 유일한 방법이 아니었다. 한 파일이어도 grep 으로 구간만 읽으면 1,928 이었다. 숫자를 바로잡는다.

그리고 E 가 실재하는 위험이다. 결정이 56개라 "하트" 같은 흔한 낱말은 13~16개 노드에 걸린다. 매칭된 것을 전부 읽으면 쪼개기 전보다 나쁘다. 구조가 아니라 규율이 이득을 만든다 — 파일명을 보고 골라 읽어야 한다.

토큰보다 중요한 것 — 경계가 의미와 일치한다

B 경로에는 숨은 결함이 있다. 결정이 어디서 시작해 어디서 끝나는지 모르므로 읽을 구간을 찍어야 한다. 짧은 결정은 다음 결정까지 달려오고, 긴 결정은 대안·영향 절이 잘린다. 잘린 줄 모르고 답한다.

노드는 파일을 열면 정확히 결정 하나가 온전히 온다. 그리고 실제로, 쪼개기 전에는 B 를 안 하고 A 를 했다 — 이번 작업 중 통째로 읽다가 시스템이 "내용이 너무 커서 포함할 수 없다"며 잘라냈다. 구조가 게으른 선택을 막는다.

그 밖의 절감

작업	전	후	절감
매 세션 자동 주입 모든 작업이 지불	10,031	8,782	-12%
표 하나 파악 네 파일 → 한 장	4파일	1장	484토큰

지금 규모

109	12	3	0
위키 노드	원본 파일	생성·검사 스크립트	끊어진 링크

토큰이 아닌 이득

무엇	어떻게 달라졌나
뒤집힌 결정이 드러난다	frontmatter 에 <code>status: superseded</code> 와 <code>superseded_by</code> 를 단다. 지금 2건 — 후보 스코프 규칙, 초대 링크 시점. 색인에서 취소선으로 보이고 후속 결정으로 화살표가 간다.
낡음을 기계가 잡는다	<code>db/doc_lint.py</code> 가 여섯 가지를 본다 — 숫자 · 사라진 이름 · 끊어진 링크 · 없는 스크립트 · 낡은 생성물 · 미반영 회의록 .
목록을 손으로 관리하지 않는다	색인과 표 페이지는 생성물이다. 스키마가 바뀌면 다시 뽑는다. 손으로 만든 목록이 어긋나 있던 사고가 반복되지 않는다.
출처가 생겼다	구 서비스 숫자(탈퇴 70,764건·잔액 20억)의 원본이 저장소 밖에 있었고 git 으로 추적되지도 않았다. 인용만 있고 근거가 없는 상태였다.
옵시디언에서 열린다	그래프 뷰·백링크·속성 필터가 그대로 작동한다. 별도 도구가 필요 없다.

04 어떻게 바꿨나

한 번에 옮기지 않고 **측정** → **분할** → **자동화** 순으로 갔다.

1. 먼저 잼다

어느 파일이 얼마를 쓰는지, 언제 읽히는지부터 확인했다. 그 결과 **이득의 90%가 DECISIONS.md 한 파일에 몰려 있다**는 것이 드러났고, 나머지 문서는 건드리지 않기로 했다.

```
DECISIONS.md 24,946 · 56항목 · 중앙값 456 → 낭비 98.2%
CLAUDE.md 10,031 · 매 세션 자동 주입
[[DECISIONS]] 19곳에서 가리킴 (2위의 다섯 배)
```

2. 결정을 노드로 쪼갬다

파일명은 **ASCII 슬러그**로 했다. 한글 파일명은 git-셸에서 깨져 읽기 어렵다. 제목은 frontmatter 안에 한글로 둔다.

DECISIONS.md 는 **색인으로 남겼다**. 그래야 기존 [[DECISIONS]] 링크 19곳이 그대로 산다. 본문 참조 9곳은 개별 노드로 다시 걸었다 — 전부 색인만 가리키면 **그래프가 별 모양이 되어** 노드로 쪼갬 의미가 없다.

3. CLAUDE.md 는 조심스럽게 덜어냈다

원칙 하나로 갔다 — 안전 규칙은 빼지 않는다.

조건부로 읽히는 규칙은 안 읽힐 수 있고, 그 순간 규칙이 없는 것과 같다. 이 프로젝트의 핵심 가치가 "조용히 틀리는 것을 막는다" 인데, 안전 규칙을 조건부 로딩으로 바꾸는 것은 정확히 그 반대다.

그래서 **빠** 절마다 **함정 이름이 든 한 줄**을 남겼다. 값은 안 읽어도 되지만 함정이 있다는 사실은 항상 보인다.

⚠ 증분 적재는 조용히 틀린다. 실제로 걸린 함정 넷 — _source 없는 조인 · 컬럼 추가 후 워터마크 정지 · 지운 행의 유형 · 과거 시각 백필.
파이프라인을 건드리기 전에 [[ops-bigquery]] 를 읽는다.

4. 생성물은 스크립트가 만든다

스크립트	줄	무엇
db/wiki_index.py	163	docs/index.md 재생성
db/wiki_tables.py	251	docs/tables/ 40장 재생성
db/doc_lint.py	222	문서 주장을 실제와 대조

wiki_tables.py 는 erd.py 의 스키마 읽기 코드를 재사용한다. DB 를 읽는 코드를 두 벌 두지 않는다.

5. lint 를 믿을 수 있게 만들었다

처음엔 어긋난 것을 전부 실패로 뒀더니 **7건이 전부 오탐**이었다. "42 → 41 테이블" 같은 이력 서술을 기계가 현재 주장과 구별하지 못한다.

늘 빨간불인 검사는 아무도 안 본다. 그래서 ★ (고쳐야 함)와 • (사람이 판단)를 갈랐다. 링크·스크립트·생성물·미반영 회의록은 기계가 확정할 수 있으므로 실패로, 숫자와 이름은 알림으로 둔다.

05 이 프로젝트에 맞춘 부분

원문 패턴을 그대로 옮기지 않았다. 상황이 하나 다르다.

karpathy 의 전제와 다른 점

원문은 "외부 문서를 계속 읽어 지식으로 컴파일한다"는 상황을 가정한다. 원본이 불변이므로 위키는 한 방향으로만 자란다.

이 저장소는 살아 있는 코드베이스다. 원본이 둘이다 —

원본	성질	낡으면
raw/ 회의록·리포트	불변	안 낡는다
코드·DB DDL·스크립트	매 커밋 바뀐다	위키가 조용히 낡는다

lint 연산이 그 두 번째 때문에 있다. 원문에는 lint 가 모순·고아 문서를 찾는 용도인데, 여기서는 코드와의 어긋남을 찾는 것이 주 목적이다.

하지 않은 것

- design-spec · ONBOARDING · TEAM-PLAN 은 쪼개지 않았다. 사람이 처음부터 끝까지 읽는 문서라, 쪼개면 읽는 흐름만 깨지고 아무도 이득을 못 본다.
- RAG·임베딩을 쓰지 않는다. 색인 파일 하나로 충분하다. 문서가 109개고, 제목과 한 줄 요약만 보면 어느 노드를 열지 정할 수 있다.

남은 비용 — 정직하게

무엇	얼마나
색인 자체가 가벼운 편은 아니다	docs/index.md 가 5,889 토큰이다. 문서 109개를 한 줄 요약과 함께 싣기 때문. 결정 조회는 DECISIONS.md (1,810) 만 보면 되지만, 전체를 탐색할 때는 이 비용을 낸다.
노드를 타고 돌면 손해가 된다	매칭된 노드를 전부 읽으면 6,907 토큰으로 쪼개기 전(1,928)보다 2.6배 나쁘다. 파일명을 보고 골라 읽는 규율이 이득의 전제다. 읽기 호출마다 오버헤드도 붙는다.
세션 전체로는 몇 % 다	실제 작업에서 토큰 대부분은 도구 출력(SQL 결과·빌드 로그·코드 읽기)이다. 문서 절감은 결정 조회 작업에 집중적으로 나타난다.

06 어떻게 쓰나

사람이 할 일은 원본을 넣는 것, 나머지는 에이전트가 한다.

처음 열었다면 — 읽는 순서

알고 싶은 것	어디로
이 프로젝트가 뭐가	README.md → docs/design-spec.md
내 손으로 돌려보고 싶다	docs/ONBOARDING.md
문서가 다 어디 있나	docs/index.md — 전체 목록
왜 이렇게 했지?	DECISIONS.md 색인 → 해당 결정 노드
이 표는 뭐지?	docs/tables/<표이름>.md — 한 장에 다 있다
최근에 무슨 일이	docs/log.md

옵시디언으로 열면 저장소 루트가 볼트다. 검색(`Ctrl+0`)으로 노드 이름을 치는 것이 가장 빠르다. 그래프는 전체를 조망할 때만 쓴다.

회의록을 남길 때

1. raw/meetings/_template.md 를 복사해 YYYY-MM-DD-주제.md 로
2. 정한 것 / 못 정한 것 / 오간 이야기 / 숫자를 적는다. **숫자에는 출처를 함께** — 나중에 문서 곳곳에 인용된다
3. 에이전트에게 **"회의록 반영해줘"** — 결정이 노드로 옮겨지고, 영향받는 문서가 고쳐지고, 색인이 갱신되고, 이력에 남는다

반영을 잊으면 db/doc_lint.py 가 ingested: false 를 잡는다.

회의 없이 결정이 났을 때

실제로는 이쪽이 더 흔하다. 작업하다 판단이 서면 그 자리에서 docs/decisions/<슬러그>.md 를 만든다.

```
---
title: 친구를 끊어도 행을 지우지 않는다      # 한글. 색인에 그대로 쓰인다
date: 2026-07-31
group: 친구                                # 12개 갈래 중 하나
status: active                             # 뒤집히면 superseded
tags: [결정, 친구]
---

**결정** 무엇을 정했나  **이유** 왜, 무엇을 근거로
**대안** 무엇을 기각했고 왜  **영향** 무엇이 따라 바뀌나
```

파일명은 ASCII 케밥케이스. 한글 파일명은 git·셸에서 깨져 읽기 어렵다.

기존 결정을 뒤집는다면 — 새 노트만 쓰고 끝내면 안 된다. **옛 노트에** `status: superseded` 와 `superseded_by` 를
단다. 뒤집힌 결정이 그대로 살아 있는 것이 이 위키에서 가장 위험한 상태다. 나중에 누군가 그것을 근거로 삼는다.

⚠ 하지 말아야 할 것

무엇	왜
생성물을 손으로 고치기 <code>docs/index.md</code> · <code>docs/tables/</code> · <code>docs/erd.md</code>	다음 생성 때 날아간다 . 고칠 것이 있으면 원본을 고친다 — 표 설명은 DDL 주석, 빈 표 사유는 <code>db/erd.py</code> 의 <code>EMPTY_REASON</code>
<code>raw/</code> 안의 파일 고치기	회의에서 정한 것이 바뀌어도 회의록은 그대로 둔다. 바뀐 것은 새 결정 으로 적는다 — 그래야 "언제 왜 마음이 바뀌었나"가 남는다
문서에 스키마·명령어 베껴 적기	DDL과 스크립트가 진실이다. 베끼면 두 번째 진실 이 생기고 반드시 어긋난다
결정을 색인에만 적기	<code>DECISIONS.md</code> 는 목록일 뿐이다. 내용은 노트 파일에 있어야 검색·연결· <code>superseded</code> 추적이 된다

스키마를 바꾼 뒤

```
python db/erd.py          # ERD 재생성
python db/wiki_tables.py  # 표 페이지 재생성
python db/doc_lint.py     # 문서가 실제와 맞는지
```

옵시디언으로 볼 때

저장소 루트를 볼트로 연다. `group` · `status` · `tags` 속성으로 필터가 되고, 그래프에서 `superseded` 노트 2개가
각각 후속 결정으로 화살표가 가 있는 것이 보인다.

다음 단계에서 바로 쓰인다 — 곧 합성 데이터를 만들 때 표 40개를 하나씩 놓고 "이 컬럼에 무슨 값이 들어가야 하고
왜 그런가"를 봐야 한다. `docs/tables/` 각 장에 **생성기가 그 표를 만드느냐**가 표시돼 있고, 지금 **14개가 미구현**이다
— 게시판 4 · 신고제재차단 3 · 추천거절 1 · 학교정보 6.